



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Студент _____ Зайковская Екатерина Владимировна

(Фамилия Имя Отчество)

Группа _____ ИУ9-62Б

Тип практики _____ производственная

Название предприятия _____ Институт Программных Систем им. А.К. Айламазяна РАН

Студент _____

(Группа)

(Подпись, дата)

(И.О. Фамилия)

Рекомендуемая оценка _____

Руководитель практики

от предприятия

(Подпись, дата)

(И.О. Фамилия)

Руководитель практики

(Подпись, дата)

(И.О. Фамилия)

Оценка _____

2025 г.

СОДЕРЖАНИЕ

1	Характеристика предприятия	3
2	Индивидуальное задание	4
3	Теоретическая часть	6
3.1	Вспомогательные утверждения	6
3.2	Применение суффиксных деревьев	9
4	Реализация	12
5	Отладка и тестирование	16
	ЗАКЛЮЧЕНИЕ	18

1 Характеристика предприятия

Производственная практика проходила в Институте программных систем Российской академии наук (кратко — ИПС им. А.К. Айламазяна РАН). Институт Программных Систем был создан в апреле 1984 года как Филиал Института проблем кибернетики АН СССР по решению Правительства СССР, направленному на развитие вычислительной техники и информатики в стране. Руководителем ФИПК АН СССР был назначен д.т.н., профессор Альфред Карлович Айламазян. В 1986 году Филиал Института проблем кибернетики был преобразован в Институт программных систем АН СССР, а в 2008 году институту было присвоено имя его первого директора профессора А.К. Айламазяна.

С момента создания основными научными направлениями деятельности института являлись:

- высокопроизводительные вычисления,
- программные системы для параллельных архитектур,
- автоматизация программирования,
- телекоммуникационные системы и медицинская информатика.

Сегодня Институт программных систем имени А.К. Айламазяна РАН объединяет пять исследовательских центров:

- Исследовательский центр мультипроцессорных систем (ИЦМС),
- Исследовательский центр медицинской информатики (ИЦМИ Интерин),
- Исследовательский центр искусственного интеллекта (ИЦИИ),
- Исследовательский центр процессов управления (ИЦПУ),
- Исследовательский центр системного анализа (ИЦСА).

Практика проходила в исследовательском центре мультипроцессорных систем, в лаборатории автоматизации программирования.

2 Индивидуальное задание

Для формулирования задания необходимо предварительно привести следующие определения.

Определение 1. Фактор-кодом будем называть множество строк $F = \{w_0, \dots, w_n\}$ такое, что

$$\forall x_1, \dots, x_k, y_1, \dots, y_k \in F \mid x_1 \dots x_k = y_1 \dots y_k \implies \forall i = 1 \dots k \quad x_i = y_i$$

Определение 2. Словарь S определяет язык $L = \mathcal{L}(S)$, если L содержит в себе те и только те слова, которые включают в себя как подстроки все слова из S (в произвольном порядке, произвольной кратности, также в слова из L могут входить и другие произвольные подстроки).

В определенных выше терминах изучим особенности задания языков через словари, решим следующие задачи:

1. Пусть дано множество строк — словарь S . Необходимо построить по нему фактор-код F такой, что $\mathcal{L}(F) = \mathcal{L}(S)$, т.е. множества S и F задают один и тот же язык.
2. Пусть даны фактор-коды F_1 и F_2 . Необходимо построить по ним эффективное представление их объединения, т.е. фактор-код языка $\mathcal{L}(F_1 \cup F_2)$.
3. Пусть даны фактор-коды F_1 и F_2 . Необходимо построить минимальный фактор-код, накрывающий $\mathcal{L}(F_1) \cup \mathcal{L}(F_2)$.

Заметим, что словарь, минимальный для своего языка с точки зрения количества входящих в него элементов, будет являться фактор-кодом.

Действительно, пусть дан словарь S , не являющийся фактор-кодом, и пусть он для его языка является минимальным. Тогда

$$\exists x_1, \dots, x_k, y_1, \dots, y_k \in S \mid (x_1 \dots x_k = y_1 \dots y_k \ \& \ \exists i = 1 \dots k \quad x_i \neq y_i).$$

Возьмем пару x_i, y_i , в которой впервые произошло несовпадение (т.е. $x_j = y_j \ \forall j = 1 \dots i - 1$). Так как строки $x_1 \dots x_k$ и $y_1 \dots y_k$ посимвольно совпадают, то x_i является префиксом y_i или наоборот. Пусть для определенности $y_i = x_i p$, тогда $S \setminus \{x_i\}$ задает тот же язык, что и S — это следует из определения словаря

(подстрока x_i автоматически содержится во всех словах языка, т.к. содержится в y_i). Получили противоречие к поставленному изначально утверждению — значит исходный словарь не мог являться минимальным.

Это рассуждение наводит на мысль, что в словарях, не являющихся фактор-кодами, могут содержаться «избыточные» строки, входящие как подстроки в другие элементы словаря. Далее, в теоретических обоснованиях, будет объяснена роль этого свойства.

В задание входят также следующие условия:

1. Для реализации алгоритма использовать размеченное обобщенное суффиксное дерево.
2. Сравнить скорость алгоритма с наивной реализацией (перебором всех подстрок).

3 Теоретическая часть

3.1 Вспомогательные утверждения

Рассмотрим подробнее пункты задания.

Для дальнейшего рассуждения необходимо сделать оговорку об алфавите, поскольку для некоторых крайних случаев формулировка утверждений, приведенных ниже, становится не такой тривиальной. Поэтому далее предполагается, что алфавит описываемых языков больше (с точки зрения мощности множеств) множества всех используемых символов в словах фактор-кодов. Также полагаем, что все словари — конечные множества, поэтому алфавит можно считать максимум счетным множеством.

1. Построение по словарю фактор-кода.

Обратимся снова к рассуждению о минимальном словаре, приведенному ранее. Из него формально сформулируем следующие утверждения.

Утверждение 1. *Если словарь S определяет язык L и при этом найдутся различные u, w из S такие, что u — подстрока w , то словарь $S' = S \setminus \{u\}$ также определяет язык L .*

Утверждение 1 следует из определения словаря.

Утверждение 2. *Удаление из произвольного словаря всех строк, которые являются собственными подстроками других элементов словаря, позволяет получить фактор-код.*

Доказательство. Проведем операцию удаления столько раз, сколько возможно. Предположим, что полученное множество не оказалось фактор-кодом.

Тогда, аналогично рассуждению в вводной части, найдутся $x_1 \dots x_k = y_1 \dots y_k$ и $x_i \neq y_i$. Из этого следует, что в множестве после удаления осталась строка, которая является подстрокой другой — противоречие доказывает утверждение. $\square$

Таким образом, для решения первой подзадачи достаточно уметь внутри некоторого множества эффективно проверять строки на включение друг в друга в качестве подстроки. Таким образом, построение фактор-кода по словарю полностью описывается утверждением 2.

2. Построение фактор-кода $\mathcal{L}(F_1 \cup F_2)$

В данном случае множество $F_1 \cup F_2$ можно рассматривать как произвольный словарь. Тогда его можно привести к виду фактор-кода так же, как в предыдущей подзадаче.

Отметим также, что $\mathcal{L}(F_1 \cup F_2) = \mathcal{L}(F_1) \cap \mathcal{L}(F_2)$ — это верно из определения словаря (все слова языка будут включать строки, входящие и в F_1 , и в F_2).

3. Построение фактор-кода $\mathcal{L}(F_1) \cup \mathcal{L}(F_2)$

Пусть $S = \{w_0, w_1, \dots, w_n\}$ — словарь, тогда для каждого слова $\alpha \in \mathcal{L}(S)$ верно, что α включает w_i как подстроку для всех i .

В обратную сторону это соотношение можно построить не для всех языков. Находясь в классе задаваемых словарем языков, их объединение зачастую не будет лежать в этом же классе. В таком случае можно лишь определить наиболее узкий язык, заданный словарем, который приближен к искомому объединению.

Пример 1. Пусть $L_1 = \mathcal{L}(\{ab, ba, cc\})$ и $L_2 = \mathcal{L}(\{bcc, cab\})$. Тогда в словах $L_3 = L_1 \cup L_2$ гарантированно будут встречаться подстроки a, b, c . Более того, т.к. ab содержится в словах L_1 , а в L_2 — cab , то в словах языка L_3 будет встречаться подстрока ab . Аналогично словарь для L_3 содержит cc .

Таким образом (пусть $dict$ — операция взятия словаря), $dict(L_3) = \{ab, cc, a, b, c\} \sim \{ab, cc\}$. Однако, как легко видеть, $L_4 = \mathcal{L}(\{ab, cc\}) \neq L_3$, т.к. $L_4 \supset L_3$, но L_4 также включает множество слов, не относящихся ни к L_1 , ни к L_2 .

Но, несмотря на это, будем здесь считать, что L_4 будет соответствовать объединению языков L_1 и L_2 в контексте языков, ассоциированных со словарем. Ниже приведем обоснование, почему такое допущение корректно.

Пусть даны два фактор-кода F_1, F_2 , цель — найти фактор-код F_3 , в некотором смысле наиболее близкий к языку $\mathcal{L}(F_1) \cup \mathcal{L}(F_2)$. Очевидно, что если некоторая строка ω содержится и в F_1 , и в F_2 , то $\omega \in F_3$. Более того, если ω — подстрока некоторой $\alpha = a_1\omega a_2 \in F_1$ и некоторой $\beta = b_1\omega b_2 \in F_2$, то $\omega \in F_3$.

Утверждение 3. Пусть F_1, F_2 — два фактор-кода. Пусть F_3 — множество, каждый элемент которого — строка, которая является общей подстрокой некоторых α и β , где $\alpha \in F_1, \beta \in F_2$.

Тогда $\mathcal{L}(F_1) \cup \mathcal{L}(F_2) \subset \mathcal{L}(F_3)$.

Доказательство. Проверим $\mathcal{L}(F_1) \subset \mathcal{L}(F_3)$ и $\mathcal{L}(F_2) \subset \mathcal{L}(F_3)$. Пусть произвольное слово $\omega \in \mathcal{L}(F_1)$, $F_1 = \{u_1, u_2, \dots, u_n\}$, $F_3 = \{p_1, p_2, \dots, p_k\}$.

При этом верно, что u_i — подстрока ω для всех i . Тогда верно, что p_i — подстрока ω для всех i — это автоматически выполняется, т.к. p_i — подстрока некоторого u_j . Так как любое слово языка $\mathcal{L}(F_1)$ также задается словарем F_3 , то $\mathcal{L}(F_1) \subset \mathcal{L}(F_3)$. Аналогичное соотношение верно для $\mathcal{L}(F_2)$. $\square$

Утверждение 4. *Определенный выше фактор-код F_3 является "наиболее точным" покрытием $\mathcal{L}(F_1) \cup \mathcal{L}(F_2)$ в том смысле, что*

$$\nexists F_4 \mid \mathcal{L}(F_1) \cup \mathcal{L}(F_2) \subset \mathcal{L}(F_4) \subset \mathcal{L}(F_3)$$

.

Доказательство. Предположим обратное. (Введем обозначение для строк ω , u : $\omega \succ u$ означает, что u — собственная подстрока ω .) Тогда, принимая во внимание, что $\mathcal{L}(F_4) \neq \mathcal{L}(F_3)$, верно, что $\exists \omega \mid \omega \in \mathcal{L}(F_3) \ \& \ \omega \notin \mathcal{L}(F_4)$. Из определения словаря получим, что $\exists u \in F_4 \mid \omega \not\succ u$. С другой стороны, из соотношения $\mathcal{L}(F_1) \cup \mathcal{L}(F_2) \subset \mathcal{L}(F_4)$ получим, что строка u содержится как подстрока во всех словах языков $\mathcal{L}(F_1)$ и $\mathcal{L}(F_2)$.

Если u не является подстрокой некоторых строк из фактор-кодов F_1 и F_2 , то невозможно гарантировать, что u содержится во всех словах $\mathcal{L}(F_1)$ и $\mathcal{L}(F_2)$. Предположим, что символ x не встречается в u . Тогда слово $\alpha \in \mathcal{L}(F_1)$, $\alpha = s_1 x s_2 \dots x s_k$ (s_i — элементы F_1) не может содержать u . Аналогичное рассуждение верно для $\mathcal{L}(F_2)$. Т.к. предполагается, что алфавит бесконечно большой, символ x всегда найдется.

Отсюда получаем противоречие, т.к. строка $u \in F_4$ должна быть подстрокой некоторых элементов фактор-кодов F_1 и F_2 , но тогда эта строка содержится в фактор-коде F_3 . $\square$

Таким образом, решение третьей подзадачи сводится к нахождению общих подстрок для двух множеств: искомый словарь будет содержать строки, которые являются максимальными общими подстроками некоторых строк из обоих данных фактор-кодов.

3.2 Применение суффиксных деревьев

Суффиксное дерево — структура данных, предназначенная для представления всех суффиксов данной строки. Каждый путь от корня к листу в суффиксном дереве соответствует одному из суффиксов. Основное преимущество суффиксного дерева заключается в том, что оно позволяет выполнять поиск подстроки за время, пропорциональное длине искомой подстроки, то есть за $O(m)$, где m — длина подстроки.

Для суффиксных деревьев существует несколько линейных алгоритмов построения, самым известным из которых является алгоритм Укконена. Реализованный в работе алгоритм МакКрейта (McCreight's algorithm) был предложен в 1976 году и стал одним из первых линейных методов [1].

Так как в поставленной задаче необходимо эффективно реализовать нахождение общих подстрок и их включения, суффиксное дерево наилучшим образом подходит под эти цели.

Построив эффективно обобщенное суффиксное дерево (generalized suffix tree, сокр. GST далее) для двух множеств, мы можем проводить операции, упомянутые выше, за время, зависящее только от длин строк.

Для построения GST использовался алгоритм МакКрейта, немного модифицированный. Для последовательного добавления строк (факторов) в дерево, согласно алгоритму, последовательно добавляются все суффиксы строки, начиная с самого длинного (см. алгоритм 1).

Листинг 1 AddFactor(F)

```
1: currentFactor  $\leftarrow F$ 
2: strIndex  $\leftarrow$  index of  $F$  in factors list
3: Add  $F$  to internal list of factors
4: Annotate(root, strIndex)
5: head  $\leftarrow$  root
6: for each suffix  $F[i..]$  do
7:   head  $\leftarrow$  AddSuffix(head, i)
8: if head  $\neq$  root then
9:   head.suffixLink  $\leftarrow$  root
```

При добавлении суффиксов запоминается внутренняя вершина (head), к которой был добавлен лист — это позволяет следующий суффикс строки добавить без обхода дерева от корня, используя переходы и свойства суффиксных ссылок.

Добавление суффикса конкретной строки сводится к проходу по дереву к месту добавления нового ребра (см. алгоритм 2) с помощью двух проходов — «быстрого» (переход по суффиксной ссылке или ее добавление) и «медленного» (последовательное сопоставление суффикса до места разветвления в дереве).

Листинг 2 AddSuffix(head, start)

Global: F, strIndex

```

1: fastHead ← FastScan(head, start)
2: newHead ← SlowScan(fastHead, start)
3: CreateLeaf(newHead, strIndex, start + newHead.depth, end = |F|-1)
4: return newHead

```

Листинг 3 FastScan(node, start)

Global: currentFactor, strIndex, root

```

1: if node = root then
2:   return root
3: if node.suffixLink exists then
4:   return node.suffixLink
5: skipped ← node.length
6: curPos ← node.start
7: if node.parent = root and skipped > 0 then                                ▷ Переход по суффиксной ссылке в корне дерева
8:   skipped ← skipped - 1
9:   curPos ← curPos + 1
10: curNode ← node.parent.suffixLink
11: AnnotateUp(curNode, strIndex)
12: nextNode ← child of curNode by character in F on curPos
13: while skipped ≥ nextNode.length do
14:   skipped ← skipped - nextNode.length
15:   curPos ← curPos + nextNode.length
16:   curNode ← nextNode
17:   Annotate(curNode, strIndex)
18:   if skipped = 0 then
19:     break
20:   nextNode ← child of curNode by character in F on curPos
21: if skipped > 0 then                                                        ▷ Разделение ребра, если осталась непрочитанная часть
22:   curNode ← SplitEdge( curNode, nextNode, nextNode.start + skipped - 1)
23: node.suffixLink ← curNode
24: return curNode

```

«Быстрый» проход моделирует прохождение по суффиксной ссылке, если она еще для данной вершины не задана (см. алгоритм 3). Здесь используется соображение, что для родительской вершины суффиксная ссылка уже задана, и

поэтому достаточно перейти по ней, а затем прочитать ребро от родительской до исходной вершины.

Полученная позиция будет вершиной, на которую будет указывать суффиксная ссылка из исходной вершины.

«Медленный» проход (см. алгоритм 4) спускается, сопоставляя посимвольно добавляемый суффикс и путь в GST до вершины, где они не совпадают. В этом и заключается все его предназначение.

Листинг 4 SlowScan(node, start)

Global: F, strIndex

```
1: curr ← node
2: AnnotateUp(curr, strIndex)
3: pos ← start + curr.depth
4: while pos < |F| and curr has child by F[pos] do
5:   child ← child by F[pos]
6:   edgePos ← 0
7:   while edgePos < child.length and F[pos] matches child edge do           ▷ сопоставляем строку на ребре с F
8:     pos++, edgePos++
9:   if edgePos = child.length then                                           ▷ если прочитано ребро до конца
10:    curr ← child
11:    Annotate(curr, strIndex)
12:   else
13:    curr ← SplitEdge(curr, child, edgePos)
14:   return curr
15: return curr
```

Алгоритмы на полученном обобщенном суффиксном дереве не представляют сложности:

1. Включение a в как подстроки (в одну из строк, хранящихся в GST) заключается в проходе по дереву с чтением a : если вершин, в которой завершается a , не листовая, то a является подстрокой, в противном случае не является.
2. Нахождение максимальной общей подстроки для двух строк — задача нахождения наиболее глубокой вершины в дереве, которая помечена индексами обеих данных строк. Этот алгоритм легко обобщается на нахождение общих подстрок, содержащихся в элементах двух множеств: достаточно отслеживать в аннотациях вершин не пару индексов, а наличие индексов строк из двух множеств одновременно

4 Реализация

Языком реализации был выбран Java [2].

Для представления обобщенного суффиксного дерева использовались три класса: GST, Node, SuperNode. Класс SuperNode представляет собой вспомогательную вершину-родитель для корневой вершины дерева, которая имитирует связь корневой вершины с самой собой по суффиксной ссылке. При этом SuperNode — наследник Node, что позволяет одинаково работать со всеми вершинами суффиксного дерева.

В реализации явно не выделены ребра — в вершине хранится информация о входящем в него ребре и дочерних вершинах. Это позволяет снизить количество операций выделения памяти.

Таким образом, поля класса Node выглядят так, как представлено в листинге 1.

Листинг 1: Класс представления вершины

```
1 public class Node {
2     @Nullable Node parent;
3     int start, end;
4     int length;
5     int depth;
6     @Nullable Node suf;
7     private @NotNull HashMap<Character, Node> children;
8
9     private @NotNull Set<Integer> annotation;
10    private @NotNull Set<Integer> finalOf;
11    int usedStrIndex;
12
13    // реализация методов
14 }
```

Для обобщенного суффиксного дерева необходимо хранить не только срез строки (start, end), которым помечено ребро, но и номер строки, из которого он взят (usedStrIndex).

Далее реализованы методы доступа к дочерним вершинам, добавление «аннотаций» — каждая вершина помечена номерами строк, по которым строится обобщенное суффиксное дерево и суффиксы которых можно прочитать при проходе через эту вершину.

Множество индексов `finalOf` является подмножеством (возможно, пустым) множества `annotation` и показывает, суффиксы каких строк могут закончиться в данной вершине. В случае явного хранения ребер с эндмаркерами `finalOf` отражало бы случаи, когда из нелистой вершины выходит ребро, помеченное только эндмаркером. Но поскольку подбор различных концевых символов для строк, по которым строится обобщенное дерево требует дополнительных затрат, был выбран способ добавления такого вспомогательного множества. Можно считать, что дерево построено со всеми различными эндмаркерами, а затем на ребрах они были отрезаны. Если в таком случае пропало ребро, то в его родителя в `finalOf` записывается индекс соответствующей строки. Если таким образом ребро к листу только лишь укоротилось, то `finalOf` просто сохраняет один индекс строки, который в данном случае будет совпадать с тем, что хранится в `annotation`.

Класс `SuperNode` реализован следующим образом (см. листинг 2).

Листинг 2: Класс представления вспомогательной вершины

```
1 public class SuperNode extends Node {
2     Node child;
3
4     public SuperNode() {
5         super(null, 0, -1, 0);
6         this.suf = this;
7     }
8
9     public void setChild(@NotNull Node child) {
10         this.child = child;
11     }
12
13     @Override
14     public boolean hasChild(Character c) {
15         return true;
16     }
17
18     @Override
19     public Node getChild(Character c) {
20         return child;
21     }
22 }
```

Так как алфавит, в котором используется дерево, не фиксирован, будем считать, что он содержит всевозможные символы. Методы имитируют существование ребер от «супер-вершины» к корню дерева по любому символу, при этом длина

таких ребер считается нулевой. Это позволяет идти вниз по дереву от SuperNode так, как будто проход происходит из корневой вершины.

Реализация класса GST объемна, поэтому отметим только некоторые особенности:

1. Как было отмечено ранее, добавление эндмаркеров не происходит, а заменяется добавлением индекса в множество финальных индексов вершины.
2. Также в каждой вершине известно, суффиксы каких строк можно прочитать, проходя через нее. Это впоследствии необходимо для нахождения максимальных общих подстрок.
3. При добавлении строк в дерево автоматически проверяется условие для построения фактор-кода. А именно, если добавляемую строку уже можно прочитать как инфикс в дереве, то она является избыточной и не добавляется. С другой стороны, если в дереве уже хранится избыточная строка, то при добавлении новой (той, которая содержит в себе эту подстроку) перезаписываются срезы на ребрах. Дополнительно избыточная строка добавляется в неиспользуемые — это происходит в тот момент, когда при добавлении некоторого суффикса новой строки полностью прочитали старую. Таким образом, избыточная строка далее не используется в дереве.

Вместо строк применялся класс Factor, который оборачивает строку и предоставляет все необходимые методы (charAt и др.).

Для представления фактор-кода использовался класс FactorCode. Он абстрагирует использование фактор-кода от особенностей хранения строк и содержит реализацию самих операций, описанных в задании.

Построение фактор-кода по словарю или по объединению словарей (задачи 1-2) сводится к операции построения обобщенного суффиксного дерева и, при необходимости, к последующему извлечению множества строк полученного фактор-кода. Последнее требует лишь не учитывать отсеянные строки из множества поданных на вход, поскольку при построении GST их индексы уже были вычислены.

Построение фактор-кода для объединения языков (задача 3), как уже известно, обращается к операции нахождения множества наибольших общих подстрок

для двух словарей. Здесь применена небольшая хитрость в реализации: чтобы в GST учесть разделение на множества, в него добавляются все строки первого словаря, а затем второго, а разделительный индекс (divider, см. листинг 3) запоминается. Тогда можно отслеживать индексы строк, сохраненные в вершинах, и точно определять, может ли прочитанная строка быть подстрокой слов и из первого, и из второго словаря.

Листинг 3: Нахождение наибольших общих подстрок

```
1  /* ...
2     методы класса GST
3  */
4  public ArrayList<Factor> LCSS(int divider) {
5      Stack<Node> stack = new Stack<>();
6      stack.push(root);
7      ArrayList<Factor> result = new ArrayList<>();
8      while (!stack.isEmpty()) {
9          Node currNode = stack.pop();
10         boolean hasDeeper = false;
11         for (Node child : currNode.getChildren().values()) {
12             if (child.hasBoth(divider)) {
13                 stack.push(child);
14                 hasDeeper = true;
15             }
16         }
17         if (currNode.hasBoth(divider) && !hasDeeper &&
18             ↪ !(currNode.equals(root)))
19             result.add(factors.get(currNode.usedStrIndex).substring(
20                 currNode.end - currNode.depth + 1, currNode.end + 1
21             ));
22     }
23     return result;
24 }
```

5 Отладка и тестирование

В процессе реализации алгоритма были написаны простые юнит-тесты (с использованием JUnit Jupiter) проверяющие на конкретных примерах следующие свойства:

- корректность построения GST путем проверки включения всех суффиксов;
- коммутативность операции объединения языков;
- ассоциативность операции объединения языков;
- коммутативность операции объединения фактор-кодов;
- ассоциативность операции объединения фактор-кодов.

Дополнительно к этому была добавлена генерация случайных наборов случайных строк с ограничением на длину. Это позволило протестировать использование суффиксного дерева для данной задачи в сравнении с представлением фактор-кода в качестве множества (set).

Для сравнения использовалась готовая эффективная реализация этой же задачи с использованием множеств.

На рисунке 1 представлены усредненные значения замеров времени вычисления 100 операций объединения языков и 100 операций пересечения фактор-кодов (в сумме) в наносекундах в зависимости от верхней границы длин строк в словарях.

Согласно проведенным измерениям, для небольших длин строк, примерно до 800 символов, реализация с помощью множеств выигрывает по времени у реализации с помощью деревьев. Затем ситуация меняется, и после этой границы можно наблюдать выигрыш алгоритмов на деревьях.

Полученные результаты можно обосновать следующими факторами:

1. Для небольших строк фактический сценарий выполнения алгоритма находится в среднем, «хорошем» временном диапазоне, поскольку в среднем наихудших случай алгоритма не достигается существенно чаще, чем в альтернативной реализации.
2. Построение суффиксного дерева требует дополнительных затрат, которые для небольших входных данных заметны.

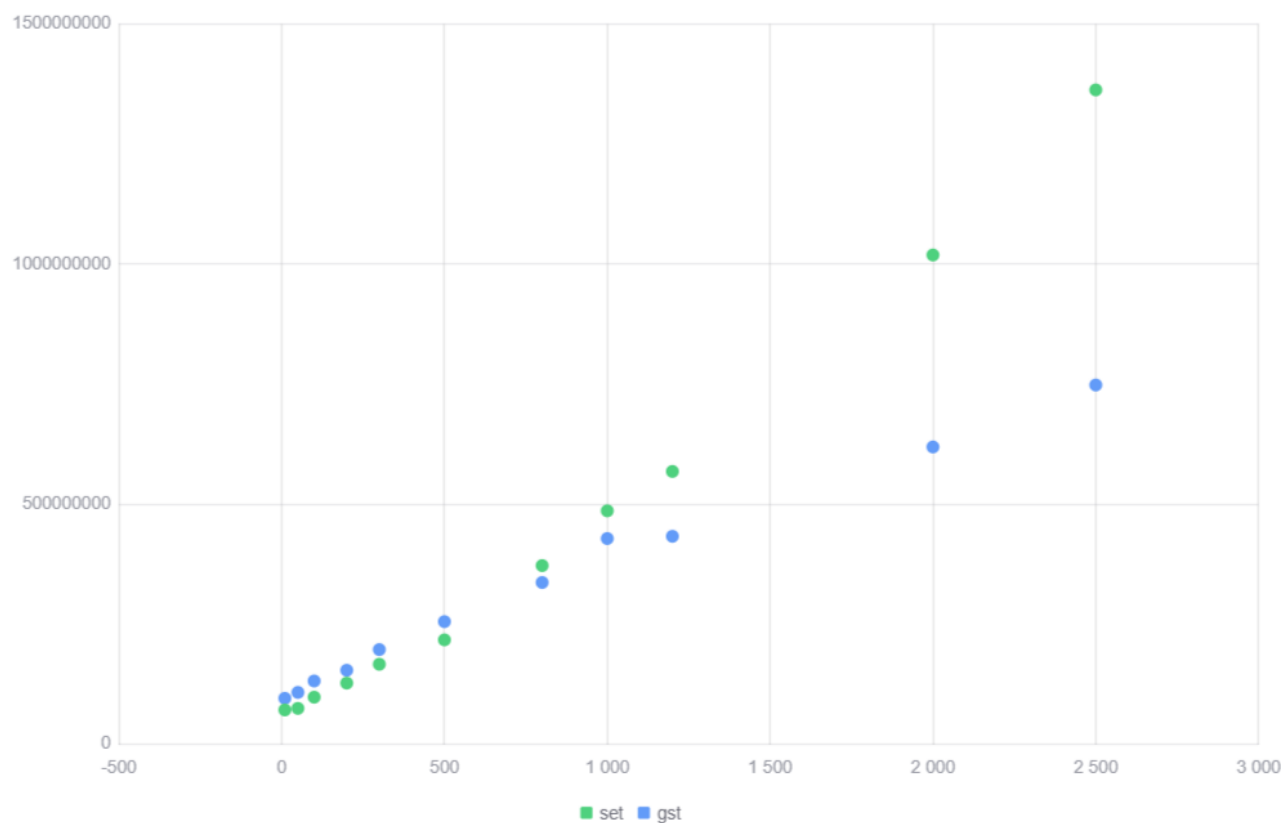


Рисунок 1 — Замеры времени в сравнении для реализации с помощью множеств и суффиксных деревьев

Дополнительно стоит отметить, что фактическое поведение алгоритма с использованием GST, основываясь на измерениях, обладает линейной зависимостью от длины строк, что подтверждает теоретическую эффективность использования суффиксного дерева.

ЗАКЛЮЧЕНИЕ

В ходе практики был реализован алгоритм на обобщенном суффиксном дереве, решающий задачу построения языков, задаваемых фактор-кодами. Эффективность реализации была сравнена с реализацией, использующей множества.

Основываясь на результатах, можно сделать вывод о том, что без оптимизаций текущая версия алгоритма на практике будет менее эффективна, чем альтернативная реализация. Хотя при увеличении входных данных на практике и сохраняется линейная зависимость времени, в инструментах анализа строк вряд ли будет необходимо работать только со строками длиной от 1000 символов и только с ними.

Например, необходимость отслеживать языки, задаваемые фактор-кодом, может понадобиться при анализе конструкций сопоставлений шаблонов в программах. Такой семантический анализ подразумевает обработку строк небольшой длины, поэтому для такого применения построение суффиксного дерева может быть избыточно.

Подводя итог, необходимо отметить, что полученную реализацию следовало бы улучшить до показателей алгоритма на множествах. Предположительно, одним из вариантов может стать оптимизация операций сравнения при проходе по дереву. Также, если возникает необходимость работы с длинными строками, можно использовать различные реализации на различных входных значениях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Algorithms on Strings, Trees, and Sequences [Электронный ресурс]. — URL: https://doc.lagout.org/science/0_Computer%20Science/2_Algorithms/Algorithms%20on%20Strings%2C%20Trees%2C%20and%20Sequences%20%5BGusfield%201997-05-28%5D.pdf ; (дата обращения: 14.07.2025).
2. Java Documentation [Электронный ресурс]. — URL: <https://docs.oracle.com/en/java/> ; (дата обращения: 14.07.2025).